



CHAPTER 2

RELATIONAL DATABASE – DESIGNING

Zakia Challal

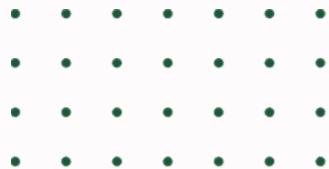
zakiachallal@gmail.com

Computer Science Engineering. 2nd year. 2024-2025



Content

- 01** Relational Database: Basic concepts
- 02** Designing relational database
- 03** Normalizing relational database
- 04** Functional Dependencies
- 05** Normal Forms
- 06** Exercise
- 07**



Introduction

- A relational database is a set of tables (relations) that store data in a structured format , following the relational model. These tables are connected through relationships, using common attributes.
- How to design these relations: which attributes in which relation and how to relate relations ?

Basic concepts

Attribute, Tuple, Cardinality, Degree

- **Attribute:** it is a column in a table that represents a specific property or characteristic of an entity. Each attribute has a **name** and a **domain** (set of allowed values).
- **Tuple:** it is a **row** in a table. It represents a **single record** containing values for each attribute.
- **Cardinality:** it refers to the **number of tuples (rows)** in a relation at a given time.
- **Degree:** it refers to the **number of attributes (columns)** in a relation.

Basic concepts

Atomic/scalar data item, Domain

- An **atomic or scalar data item** is the smallest semantic unit of data.
It is a data item with no internal structure.
- A **domain** refers to the set of all possible **scalar values of the same type** that an attribute (column) can take. It defines the data type and constraints for a specific column in a table.

Basic concepts

Relation, Intension, Extension, Schema, Instance

- A **relation** in a relational database is formally defined as:

$R(A1:D1, A2:D2, \dots, An:Dn)$ where:

- R is the name of the relation (table).
- $A1, A2, \dots, An$ are attributes (columns).
- $D1, D2, \dots, Dn$ are domains, specifying the set of valid values each attribute can take.

- A relation consists of:

Schema (Intension): The structure of the relation, including attribute names and data types.

Instance (Extension): The actual data (set of tuples) stored in the table at a given time.

Basic concepts

Integrity constraint

- An **integrity constraint** is a set of rules that ensure the coherence of data in a relational database. These constraints prevent invalid data entry and maintain the logical correctness of relationships between tables.

● Constraint Type	Purpose
Domain Constraint	Restricts values based on data type and rules
Entity Integrity	Ensures every table has a unique identifier
Referential Integrity	Maintains valid relationships between tables
Unique Constraint	Ensures uniqueness of certain attributes
NOT NULL Constraint	Prevents missing (NULL) values
CHECK Constraint	Imposes additional rules on column values

Basic concepts

Candidate key, Super key, Primary key, Foreign Key

- **Candidate key** : is a minimal set of attributes that **uniquely identifies each tuple** in a relation. A table can have multiple candidate keys.
- **Super key** : is a set of attribute that include at least one candidate key. A **superkey** may contain extra attributes that are not necessary for uniqueness.
- **Primary key** : it is a candidate key that is chosen to uniquely identify rows in a table. It cannot contain NULL values and must be unique for all records.
- **Foreign Key**: it is an attribute in one table that references the primary key of another table. It ensures referential integrity, meaning a value in the foreign key must exist in the referenced table.

Basic concepts

Relation's properties

Relations have some important properties arising from the definition:

- A relation cannot contain duplicate tuples.
- Tuples are unordered.
- Attributes are unordered.
- All attribute values are atomic.

Exercise

For each relation, define:

- The intension,
- The extension,
- The degree,
- The cardinality,
- The candidate keys,
- The primary key,
- The foreign key
- The schema of the database.

Employees			
Emp_ID	Emp_Name	Salary	Department
4182	A. Singh	36,003.12	Development
5542	M. Lee	39,818.15	Development
3351	T. Esterhazy	31,919.18	Development
6164	A. Barraclough	38,002.05	Marketing
9095	N. Prabakar	36,003.12	Research

Departments	
Department	Budget
Development	500,000
Marketing	150,000
Research	100,000

Designing a relational database

- Database design can be done using 2 methodologies:
 - Top-down: start with some relations and refine the decomposition until all requirements are met.
 - Bottom-up: start with basic relationships between attributes and build up relations
- Anomalies in database design: redundancy, incoherence.
- Find a way to evaluate the quality of a relational schema
- Normalization through defining dependencies between attributes => **Functional Dependency.**

Functional dependencies

Definition

- A functional dependency (FD) is a constraint between two sets of attributes in a relation.
It specifies that the value of one attribute (or a set of attributes) uniquely determines the value of another attribute.
- We denote: $X \rightarrow Y$ (X determine Y). X, Y are attributes or set of attributes.
- $X \rightarrow Y$ means:
If two tuples have the same value for X, they must have the same value for Y.
- X is called the determinant, and Y is the dependent.
X is called the left-hand side (LHS) and Y the the right-hand side (RHS)

Functional dependencies

Example

- Relational database....



Functional dependencies

Trivial dependency

- A **trivial functional dependency** is a dependency where the **right-hand side (Y)** is **a subset of the left-hand side (X)**. This means that the dependent attribute(s) is **already included** in the determinant.
- Mathematically, a functional dependency $X \rightarrow Y$ is **trivial** if: $Y \subseteq X$

Functional dependencies

Closure

- The **closure** of a set of functional dependencies is the **set of all functional dependencies** that can be **logically inferred** from a given set of functional dependencies **F** using inference rules.
- **Inference rules** are rules allow us to derive new dependencies from a given set. These rules, known as **Armstrong's Axioms**.
- The **closure** of a set of functional dependencies is denoted as **F⁺**

Functional dependencies

Inference rules-Armstrong's Axioms

- **Reflexivity:** if Y is a subset of X then $X \rightarrow Y$ (trivial FD)
- **Augmentation:** if $X \rightarrow Y$ then $XZ \rightarrow YZ$
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

Functional dependencies

Inference rules-more rules

More rules can be derived from Armstrong's Axioms:

- **Decomposition:** if $A \rightarrow BC$ then $A \rightarrow B$ and $A \rightarrow C$
- **Composition :** if $A \rightarrow B$ and $C \rightarrow D$ then $AC \rightarrow BD$
- **Union:** if $A \rightarrow B$ and $A \rightarrow C$ then $A \rightarrow BC$
- **Pseudo-transitivity:** if $A \rightarrow B$ and $CB \rightarrow D$ then $CA \rightarrow D$

Functional dependencies

Inference rules- Example

$R(A, B, C, D, E, F)$ and $F = \{ A \rightarrow BC, B \rightarrow E, CD \rightarrow EF \}$

We can deduce $AD \rightarrow F$ is an element of FD closure.

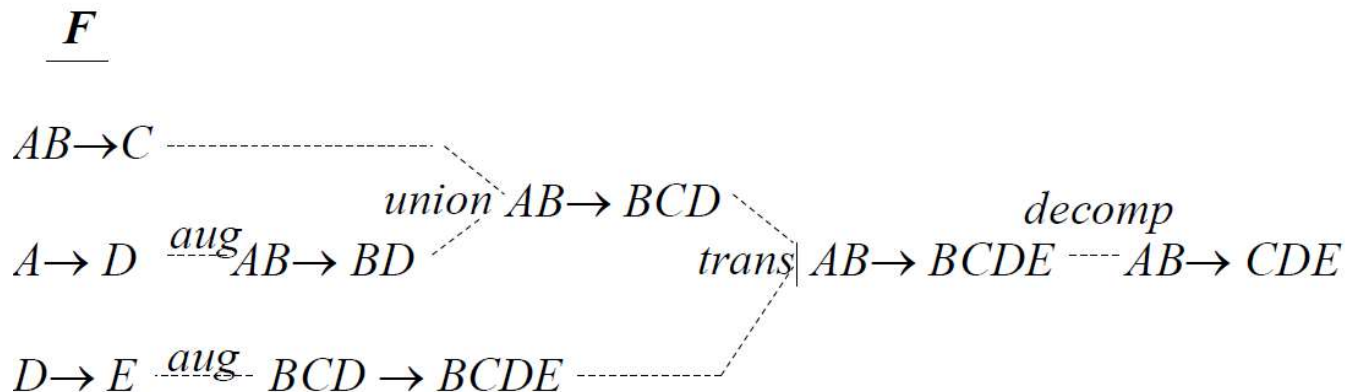
1. $A \rightarrow BC$ (given)
2. $A \rightarrow C$ (decomposition)
3. $AD \rightarrow CD$ (augmentation)
4. $CD \rightarrow EF$ (given)
5. $AD \rightarrow EF$ (transitivity)
6. $AD \rightarrow F$ (decomposition)

So, $AD \rightarrow F$ is an element of FD closure.

Functional dependencies

Closure- Example

Generating F^+



Thus, $AB \rightarrow BD$, $AB \rightarrow BCD$, $AB \rightarrow BCDE$ and $AB \rightarrow CDE$ are all elements of F^+ (part-of, there are other FDs: $AC \rightarrow CD$, $AE \rightarrow ED$, etc.)

=> Computing closure is very costly procedure.

Exercise

Consider the following **Employee** relation:

Emp_ID	Name	Dept_ID	Dept_Name	Manager_ID	Salary
101	Alice	D1	HR	M1	5000
102	Bob	D2	IT	M2	6000
103	Charlie	D1	HR	M1	5500
104	Dave	D3	Finance	M3	7000
105	Eve	D2	IT	M2	6200

- Identify **possible functional dependencies (FDs)** in the relation.
- Find the **candidate keys** for the relation.

Functional dependencies

Attribute Closure

- **Attribute closure** of a set of attributes X , with respect to a set of functional dependencies F , is the set of all attributes, A , such that $X \rightarrow A$ is deduced from F .
- **Attribute closure** determines all attributes that can be functionally derived from a given set of attributes using a set of functional dependencies (FDs).
- **Attribute closure** of a set of attributes X is denoted X^+

Functional dependencies

Computing the Attribute Closure

Input: a relation schema R , a set of functional dependencies F , a set of attributes X .

Output: The closure of X denoted X^+ .

Begin

$X^+ = \{X\}$ (include all attributes in X initially)

Repeat

for each functional dependency $A \rightarrow B$ in F do

if $A \subseteq X^+$ then $X^+ = X^+ \cup \{B\}$

end for;

Until no more attributes can be added to X^+ ;

End;

Functional dependencies

Computing the Attribute Closure – Example 1

Consider a relation $R(A,B,C,D,E)$ with the following functional dependencies:

$F=\{A \rightarrow B, B \rightarrow C, C \rightarrow D, AD \rightarrow E\}$. Find the closure of A^+ .

Start with $A^+=\{A\}$.

Apply FDs: $A \rightarrow B$, $A \in A^+$ then Add B to A^+ , $A^+=\{A,B\}$.

$B \rightarrow C$, $B \in A^+$ then Add C to A^+ , $A^+=\{A,B,C\}$.

$C \rightarrow D$, $C \in A^+$ then Add D to A^+ , $A^+=\{A,B,C,D\}$.

$AD \rightarrow E$, $\{A,D\} \subseteq A^+$ then Add E to A^+ , $A^+=\{A,B,C,D,E\}$.

No more iteration is needed since no more attribute can be added to A^+ .

Final closure: $A^+=\{A,B,C,D,E\}$

Functional dependencies

Computing the Attribute Closure – Example 2

Consider a relation $R(A,B,C,D,E, F)$ with the following functional dependencies:

$G=\{B \rightarrow E, A \rightarrow BC, CD \rightarrow EF\}$. Find the closure of B^+ .

Start with $B^+=\{B\}$.

1st iteration: $B \rightarrow E$, $B \in B^+$ then Add E to B^+ , $B^+=\{B,E\}$.

$A \rightarrow BC$, $A \notin A^+$

$CD \rightarrow EF$, $CD \notin A^+$

$B^+=\{B,E\}$

2nd iteration: $B \rightarrow E$, $B \in B^+$ then Add E to B^+ , $B^+=\{B,E\}$.

$A \rightarrow BC$, $A \notin A^+$

$CD \rightarrow EF$, $CD \notin A^+$

$B^+=\{B,E\}$, B^+ didn't changed so stop.

Functional dependencies

Applications of Attribute Closure

● Candidate Key Determination:

- If X^+ contains all attributes of a relation R , then X is a **superkey**.
- If X is irreducible (can't be reduced to a smaller set of attribute), then X is a **candidate key**.

● Checking Functional Dependency:

- To check if $X \rightarrow Y$ holds, compute X^+
- If $Y \subseteq X^+$, then $X \rightarrow Y$ holds.

● Normalization:

See later.

Functional dependencies

Equivalent sets of FDs

Two sets of functional dependencies **F** and **G** are **equivalent** if they **imply the same set of functional dependencies**. This means that every functional dependency derivable from F must also be derivable from G, and vice versa.

Formally:

F and G are equivalent if:

$$F^+ = G^+$$

Functional dependencies

Cover

- Let R be a relation and F, G two sets of functional dependencies (FDs) defined on R .
 F is a cover of G if and only if $G^+ \subset F^+$ (i.e., F implies all FDs in G).
- **Note 1:** To show that F is a cover of G , it is sufficient to prove that every functional dependency in G belongs to F (i.e., it can be derived from the FDs in F).
- **Note 2:** F and G are equivalent if F covers G and G covers F .
($F^+=G^+$ means $F^+ \subset G^+$ and $G^+ \subset F^+$ which means that F covers G and G covers F)

Functional dependencies

Cover - Example

Consider a relation $R(A, B, C, D, E, F)$

$F1 = \{ (1) A \rightarrow BC, (2) B \rightarrow D \}$

$F2 = \{ (3) A \rightarrow BD, (4) AB \rightarrow C, (5) B \rightarrow D \}$

Show that $F2$ covers $F1$.

- is (1) $A \rightarrow BC$ derivable from $F2$?

Decomposition on (3) $A \rightarrow BD$ gives (6) $A \rightarrow B$ and (7) $A \rightarrow D$

Pseudo-transitivity on (6) and (4) gives (8) $A \rightarrow C$

Union of (6) and (8) gives (1) $A \rightarrow BC$ so FD (1) is derivable from $F2$ ($\in F2^+$)

- (2) is the same as (5) so FD (2) is derivable from $F2$ ($\in F2^+$)

We conclude that $F2$ covers $F1$.

Functional dependencies

Minimal Cover

- A **minimal cover** for a set of functional dependencies (FDs) F is an equivalent set of FDs that is minimal in the sense that:
 1. Each FD has a single attribute on the right-hand side.
 2. There are no unnecessary attributes in the left-hand side.
 3. There are no redundant functional dependencies (i.e., removing any FD from the set changes the closure F^+).

Functional dependencies

Computing the Minimal Cover

Given a set of functional dependencies F , follow these steps:

Step 1: Decomposition of Right-Hand Side

- Replace each FD $X \rightarrow A_1A_2..A_n$ by $X \rightarrow A_1, X \rightarrow A_2..., X \rightarrow A_n$ ($A_1,...A_n$ are single attribute) .
- Example: $A \rightarrow BC$ becomes $A \rightarrow B, A \rightarrow C$.

Step 2: Remove extraneous attributes

- An attribute A in X (from an FD $X \rightarrow Y$) is **extraneous** if $(F - \{X \rightarrow Y\}) \cup \{(X - A) \rightarrow Y\}$ gives the same closure F^+ .
- Example: If $AB \rightarrow C$ and $A \rightarrow C$ are both in F , then B is **extraneous** in $AB \rightarrow C$, so it can be reduced to $A \rightarrow C$.

Step 3: Remove redundant FDs

- Check if an FD is implied by the rest of F . If so, remove it.
- Example: If $A \rightarrow B$, $B \rightarrow C$, and $A \rightarrow C$ exist, then $A \rightarrow C$ is redundant (since it can be derived from $A \rightarrow B$ and $B \rightarrow C$).

Functional dependencies

Computing the Minimal Cover - Example

Consider a relation $R(A, B, C, D)$ and $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C, AC \rightarrow D\}$

Compute the minimal cover of F .

Step 1: Decomposition of Right-Hand Side

- $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C, AC \rightarrow D$

Step 2: Remove extraneous attributes

- C can be eliminated from $AC \rightarrow D$: pseudo-transitivity on $A \rightarrow C$ and $AC \rightarrow D$ gives $A \rightarrow D$.
- $AB \rightarrow C$ can be eliminated: it can be deduced by augmentation of $A \rightarrow C$ that gives $AB \rightarrow CB$ and then apply decomposition which gives $AB \rightarrow C$.

Step 3: Remove redundant FDs

- $A \rightarrow C$ is implied by the functional dependencies $A \rightarrow B$ and $B \rightarrow C$, so it can be eliminated.

Exercise

Scenario: University Database

A university maintains a database with the following relation schema:

Student(StudentID, Name, Department, CourseID, Instructor, Email)

The following functional dependencies (FDs) hold in the schema:

F = {StudentID → Name, Email, Department; CourseID → Instructor, Department; Name → Email}

1. Compute the **attribute closure** of the following attribute sets: StudentID⁺, CourseID⁺, Name⁺

Consider the following alternative set of functional dependencies:

F' = {StudentID → Name, Email; StudentID → Department; CourseID → Instructor; CourseID → Department; Name → Email}

2. Are F and F' **equivalent**?
3. Find a **minimal cover** for the given set of functional dependencies F.
4. Find the **candidate keys** of the relation based on the minimal cover.

Normalization

- When designing a relational database, the first step is to define its relational schema. There are multiple possible schemas, and it is essential to choose the best ones.
- In 1971, Codd introduced the foundations of a **normalization** approach that leads to **decomposing a relation into multiple smaller, normalized schemas without any loss of information.**
- The goal is to **avoid redundancies** without causing any loss of information.

Normalization - Example

Student Table - Without Normalization

Student ID	First Name	SSN	Course	Duration	Instructor	Contact
1001	S1	CA245	Java	60 Days	Peter	617145
1002	S2	WA465	Python	45 Days	Smith	100023
1003	S3	AR365	Python	45 Days	Smith	100023
1004	S4	WA764	Java	60 Days	Peter	617145
1005	S5	CA453	C++	55 Days	Rick	454510
1006	S6	CA924	Java	60 Days	Peter	617145

Three Anomalies:

- **Update anomaly:** due to redundancy. If we need to modify the duration of the course Java to 45 days, multiple tuples must be updated.
- **Insert anomaly:** We can't add a Course until it is taken by at least one student.
- **Delete anomaly:** If we delete all the students taking the course Java, we lose information about that course.

Normalization - Example

Student Table - Without Normalization

Student ID	First Name	SSN	Course	Duration	Instructor	Contact
1001	S1	CA245	Java	60 Days	Peter	617145
1002	S2	WA465	Python	45 Days	Smith	100023
1003	S3	AR365	Python	45 Days	Smith	100023
1004	S4	WA764	Java	60 Days	Peter	617145
1005	S5	CA453	C++	55 Days	Rick	454510
1006	S6	CA924	Java	60 Days	Peter	617145

Normalization

Student Table

Student ID	First Name	SSN	Course ID
1001	S1	CA245	CS 1
1002	S2	WA465	CS 2
1003	S3	AR365	CS 2
1004	S4	WA764	CS 1
1005	S5	CA453	CS 3
1006	S6	CA924	CS 1

Course Table

Course ID	Course	Duration	Instructor ID
CS 1	Java	60 Days	PRG 1
CS 2	Python	45 Days	PRG 2
CS 3	C++	55 Days	PRG 3

Instructor Table

Instructor ID	First Name	Contact
PRG 1	Peter	617145
PRG 2	Smith	100023
PRG 3	Rick	454510

Decomposition

Definition

- Decomposition is the process of breaking a relation into multiple smaller relations to **eliminate redundancy**, improve data integrity, and achieve a higher level of normalization.
- The decomposition must be **lossless** and **dependency-preserving**:
 - **Lossless**: We should be able to reconstruct the original relation from the decomposed relations **without loss of information**.
 - **Dependency-preserving**: The functional dependencies in the original relation should be preserved in the decomposed relations

Decomposition

Lossless- Heath's Theorem

- Heath's theorem helps us **split a relation into two smaller relations without losing information** (lossless decomposition).
- If we have a relation $R(A,B,C)$ with a functional dependency: $A \rightarrow B$
Then, we can **split R into two smaller relations**: $R_1(A,B)$ and $R_2(A,C)$
This split is lossless if and only if A is a **superkey** for at least one of the decomposed relations.
- **Checking for Losslessness**: The decomposition is **lossless** if a **natural join** can perfectly reconstruct the original relation.

Decomposition

Lossless- Example

$R(\text{StudentID}, \text{Name}, \text{Course}, \text{Instructor})$

StudentID	Name	Course	Instructor
101	Alice	DBMS	Dr. Smith
102	Bob	OS	Dr. Brown
103	Carol	DBMS	Dr. Smith

$F = \{ \text{StudentID} \rightarrow \text{Name}, \text{Course} \rightarrow \text{Instructor} \}$

Choose the FD that introduces redundancy in the relation.

• **Course Information** $\rightarrow R_1(\text{Course}, \text{Instructor})$

• **Student Information** $\rightarrow R_2(\text{StudentID}, \text{Name}, \text{Course})$

Table 1: Student Table $R_1(\text{StudentID}, \text{Name}, \text{Course})$

StudentID	Name	Course
101	Alice	DBMS
102	Bob	OS
103	Carol	DBMS

Table 2: Course Table $R_2(\text{Course}, \text{Instructor})$

Course	Instructor
DBMS	Dr. Smith
OS	Dr. Brown

Decomposition

Dependency preserving

- A decomposition of a relation R into a set of sub-relations $R_1, R_2, \dots, R_n$ is said to be **dependency preserving** if the **functional dependencies of R** can be found in the decomposed relations.
- Let:
 - R be a relation with a set of **functional dependencies F** .
 - $R_1, R_2, \dots, R_n$ be a **decomposition** of R .
 - F_i be the set of **functional dependencies** relevant to R_i .
 - $F' = F_1 \cup F_2 \cup \dots \cup F_n$ (the union of the functional dependencies of all decomposed relations).
 - The decomposition is **dependency preserving** if and only if: $F^+ = (F')^+$
- This means that all functional dependencies in F^+ (from the original relation) must be **implied by $(F')^+$** , the functional dependencies in the decomposed relations.

Normal Forms

Normal Forms are **guidelines** used in database design to **reduce redundancy** and **improve data integrity** by **decomposing** based on **functional dependencies**.

Normal Form	Main Goal	Eliminates
1NF (First Normal Form)	Ensure atomic values	Repeating groups (arrays, lists in a single column)
2NF (Second Normal Form)	Remove partial dependencies	Attributes depending on part of a composite key
3NF (Third Normal Form)	Remove transitive dependencies	Attributes depending on non-key attributes
BCNF (Boyce-Codd Normal Form)	Ensure every determinant is a superkey	Some functional dependency issues
4NF (Fourth Normal Form)	Remove multivalued dependencies	Redundant independent relationships

First Normal Form (1FN)

– No Repeating Groups

- A relation is in 1NF if:
 - All attributes contain **atomic values** (no lists, arrays, or sets).

♦ Example (Non-1NF Table):

StudentID	Name	Courses
101	Alice	Math, CS
102	Bob	Physics

✗ Issue: **Courses** contain multiple values.

♦ Conversion to 1NF:

StudentID	Name	Course
101	Alice	Math
101	Alice	CS
102	Bob	Physics

✓ Fixed by storing atomic values in separate rows.

Second Normal Form (2FN)

– No Partial Dependencies

- A relation is in 2NF if:
 - It is in **1NF**.
 - **All non-key attributes fully depend on the entire primary key** (not just part of it).

♦ Step 1: Non-2NF Table (1NF)

Consider a **Class Enrollment** table that includes **StudentID**, **Course**, **StudentName**, and **Marks**:

StudentID	Course	StudentName	Marks
1	Math	Alice	90
1	Science	Alice	85
2	Math	Bob	88

✓ **Primary Key:** (StudentID, Course)

✗ **Issue:** StudentName depends only on StudentID, not the whole key.

♦ Step 2: Convert to 2NF

✓ Split into two tables to remove the partial dependency.

1 Students Table (StudentID → StudentName)

StudentID	StudentName
1	Alice
2	Bob

2 Enrollments Table (StudentID, Course → Marks)

StudentID	Course	Marks
1	Math	90
1	Science	85
2	Math	88

Third Normal Form (3FN)

– No Transitive Dependencies

● A relation is in 3NF if:

- It is in **2NF**.
- **No attribute depends on a non-key attribute** (i.e., no transitive dependency).

♦ Example (Non-3NF Table):

StudentID	Department	DepartmentHead
101	CS	Dr. Allen
102	Math	Dr. Baker

✗ Issue:

- DepartmentHead depends on Department, not directly on StudentID.

♦ Conversion to 3NF:

Students Table (StudentID → Department)

StudentID	Department
101	CS
102	Math

Departments Table (Department → DepartmentHead)

Department	DepartmentHead
CS	Dr. Allen
Math	Dr. Baker

✓ Now, all attributes depend only on the primary key.

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm

Input

- A relation schema R with a set of attributes A .
- A set of functional dependencies F on A

Output

- A decomposition $D = \{R_1, R_2, \dots, R_n\}$ of R such that:
 1. Each R_i is in 3NF.
 2. The decomposition is lossless.
 3. The decomposition is dependency-preserving.

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm

Algorithm Steps

- **Step 1: Compute a Minimal Cover F_m of F**

- **Step 2: Construct Relations Based on F_m**

For each functional dependency $X \rightarrow Y$ in F_m , create a relation: $R_i = XUY$

- **Step 3: Eliminate Redundant Relations**

If a relation R_i is a subset of another relation R_j (i.e., $R_i \subseteq R_j$), discard R_i .

- **Step 4: Ensure a Superkey is Present**

Compute a superkey K of the original relation R .

If none of the generated relations contain a superkey of R , introduce an additional relation: $R_k = K$.

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm- Example

$R(\text{StudentID}, \text{Name}, \text{CourseID}, \text{CourseName}, \text{InstructorID}, \text{InstructorName})$

$F = \{\text{StudentID} \rightarrow \text{Name}, \text{CourseID} \rightarrow \text{CourseName}, \text{InstructorID}, \text{InstructorID} \rightarrow \text{InstructorName}\}$

Step 1: Compute a Minimal Cover F_m

- **Breaking FDs into Single Attributes**

$\text{CourseID} \rightarrow \text{CourseName}, \text{InstructorID}$

Split into: $\text{CourseID} \rightarrow \text{CourseName}$ $\text{CourseID} \rightarrow \text{InstructorID}$

- **Remove non necessary attribute in the left side**
- **Remove redundant FD**

The minimal cover:

$F_m = \{\text{StudentID} \rightarrow \text{Name}, \text{CourseID} \rightarrow \text{CourseName}, \text{CourseID} \rightarrow \text{InstructorID}, \text{InstructorID} \rightarrow \text{InstructorName}\}$

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm- Example

Step 2: Construct Relations Based on Fm

Each FD forms a **relation**:

R1(StudentID, Name)

R2(CourseID, CourseName)

R3(CourseID, InstructorID)

R4(InstructorID, InstructorName)

Step 3: Eliminate Redundant Relations

No redundant relation.

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm- Example

Step 4: Ensure a Superkey is Present

Calculate a superkey for R:

$(\text{StudentID}, \text{CourseID})^+ = \{\text{StudentID}, \text{Name}, \text{CourseID}, \text{CourseName}, \text{InstructorID}, \text{InstructorName}\}$

Since **(StudentID, CourseID)** is a **superkey** and none of the relation R_i contain a super key, we add:

R5(StudentID, CourseID)

Normalization into 3NF

Bernstein's 3NF Synthesis Algorithm- Example

Final 3NF schema:

R1(StudentID, Name)

R2(CourseID, CourseName)

R3(CourseID, InstructorID)

R4(InstructorID, InstructorName)

R5(StudentID, CourseID)

Relation	Attributes
Student(StudentID, Name)	Student Info
Course(CourseID, CourseName)	Course Info
Teaches(CourseID, InstructorID)	Course-Instructor Mapping
Instructor(InstructorID, InstructorName)	Instructor Info
Enrollment(StudentID, CourseID)	Student-Course Mapping

Boyce-Codd Normal Form (BCNF)

- Every determinant is a superkey

● A relation is in BCNF if:

- It is in **3NF**.
- For each FD: $X \rightarrow Y$, X must be a superkey.

Student	Teacher	Subject
Shayan	R.Das	Database
Shayan	K.Raman	C
Tahira	R.Das	Database
Tahira	N.Gupta	C

FD: { (student, Teacher) $\rightarrow$ subject, (student, subject) $\rightarrow$ Teacher, (Teacher) $\rightarrow$ subject }

Candidate keys are (student, teacher) and (student, subject).

R1		R2	
Teacher	Subject	Student	Teacher
R.Das	Database	Shayan	R.Das
K.Raman	C	Shayan	K.Raman
N.Gupta	C	Tahira	R.Das
		Tahira	N.Gupta



The END